# LOVELY PROFESSIONAL UNIVERSITY

## SCHOOL OF COMPUTER SCIENCE

---

**ACADEMIC TITLE: CA2 (PROJECT REPORT)**

# PROJECT TITLE: HEALTH ANALYSIS AND DISORDER RISK PREDICTION USING APACHE PYSPARK

*Distributed Multi-Class Machine Learning Workflow on Apache Spark Cluster Environment*


**NAME OF THE STUDENT-:** Mandhala Yashwanth Reddy

**REGISTRATION NUMBER-:** 12304480

**ROLL NUMBER-:** 36


**COURSE CODE-:** INT315

**COURSE TITLE-:** CLUSTER COMPUTING

**NAME OF THE FACULTY -:** PARDEEP KUMAR

**GIT HUB LINK -:**

https://github.com/yashwanthreddy13/Health-Analysis

# TABLE OF CONTENTS


| Section / Topic | Page |
|---|---|
| **1. Title Page & Academic Compliance** | **1** |
| **Table of Contents** | **2** |
| **2. Problem Statement** | **3** |
| 2.1 The Clinical & Epidemiological Context | 3 |
| 2.2 Big Data & Computational Bottlenecks in Single-Node Processing | 3 |
| 2.3 Role of Distributed Cluster Computing (Apache Spark) | 3 |
| **3. Objectives of the Project** | **3** |
| **4. Dataset Description** | **4** |
| 4.1 Overview of the 100,000-Record Cohort | 4 |
| 4.2 Table 1: Dataset Feature Taxonomy & Attributes | 4 |
| 4.3 Target Distribution & Class Imbalance Profile | 4 |
| **5. Dataset Source and Reference** | **4** |
| **6. Tools and Technologies Used** | **5** |
| 6.1 Table 2: Software Architecture & Role Breakdown | 5 |
| 6.2 Single-Node Multi-Threaded Cluster (local[*]) vs. Multi-Node Clusters | 5 |
| **7. Implementation and Methodology** | **5** |
| 7.1 End-to-End Workflow & Operational Phases (Steps 1 to 8) | 5 |
| 7.2 Target Leakage Prevention Rationale | 6 |
| 7.3 Mathematical Formulations of Implemented Evaluation Metrics | 6 |
| **8. Complete Source Code** | **7** |
| Listings 1 to 3: Environment Setup, Ingestion, Missing Values, and EDA | 7 |
| Listings 4 to 5: Feature Selection, Pipeline, Model Training, and Evaluation | 8 |
| **9. Execution Screenshots** | **9** |
| Figures 1 to 2: Environment Setup & Dataset Upload | 9 |
| Figures 3 to 4: Schema Loading & Missing Value Verification | 10 |
| Figures 5 to 7: Pipeline Preprocessing, Model Fitting, and Test Predictions | 11 |
| **10. Output and Result Screenshots** | **12** |
| Figures 8 to 10: Target Distribution, Bar Chart, and Biomarker Summary Table | 12 |
| Figures 11 to 14: 2x2 Comparative Grid of Clinical Biomarkers | 13 |
| Figures 15 to 16: Multiclass Evaluation Console & Performance Bar Plot | 14 |
| **11. Result Analysis and Discussion** | **15** |
| 11.1 Table 3: Summary of Measured Model Evaluation Metrics | 15 |
| 11.2 Discussion of Classification Accuracy and Class Imbalance | 15 |
| 11.3 Analysis of Physiological Gradients and Clinical Significance | 15 |
| 11.4 Cluster Computing Performance Observations (Catalyst, Caching, Parallelism) | 16 |
| 11.5 Limitations of the Current Implementation | 16 |
| **12. Conclusion and Future Scope** | **16** |
| **13. References** | **16** |

## 2. Problem Statement

**2.1 Clinical Background and Health Impact:** Clinical Background and Health Impact: Sleep quality plays a role in how the body maintains metabolic balance supports immune health and regulates heart and blood pressure. In studies conditions like chronic insomnia obstructive sleep apnea and circadian rhythm disorders are common. These affect than 30 percent of adults worldwide [1]. When these sleep problems go unnoticed they can lead to long-term health issues. These include blood pressure, type-2 diabetes, heart disease and problems with memory, focus and alertness during the day. The reliable way to diagnose sleep disorders is through an overnight Polysomnography test or PSG. This test is done in sleep labs. Involves many sensors attached to the body to monitor brain activity, breathing, heart rate and movement.. P sg comes with big limitations. It requires wires, a stay in a hospital or clinic high costs and long delays in getting an appointment. Beca use of these challenges it's not practical to use PSG for screening groups of people regularly. That's why there is a need for non-invasive tools that can assess sleep-related risks using everyday data like activity levels, heart rate and sleep patterns collected through wearable devices.

**2.2 Data Volume and Single-Node Memory Limits:** Today devices such as fitness trackers and smartwatches along with electronic health records generate huge amounts of health data. These include heart rate during the night sleep stages daily step counts and stress levels. This constant flow of information helps doctors monitor health early and prevent illness.. It also creates massive datasets that are hard to manage. In our project we have a group of 100,000 patients each with data across 32 measurements. Standard tools in Python like Pandas and Scikit-Learn are built for one computer at a time. They load the dataset into the computer's memory and run tasks one after another on a single CPU. When we try to process numbers of records—especially with tasks like encoding categorical variables creating feature vectors or training complex machine learning models—this single-node setup uses too much memory. The system starts swapping data to disk which slows everything down. Training times grow longer and longer. The system fails to keep up as the data grows into the millions of records.

**2.3 Motivation for Distributed Processing with Apache Spark:** To handle these data challenges we use distributed computing systems. Apache Spark is one framework that can run across many computers at once. It breaks datasets into smaller parts called partitions and spreads them across multiple CPU cores. This allows tasks to run simultaneously cutting down processing time. Spark includes an engine called Catalyst that plans the most efficient way to run operations. It also uses a Directed Acyclic Graph or DAG to delay steps until they are truly needed. By building our machine learning pipeline with PySpark and Spark MLlib we can work with data at any scale—whether on a computer with several cores or a large cluster, with many machines. The core logic stays the same no matter the size making it easy to scale up without rewriting the code.

## 3. Objectives of the Project

Our main goal is to build, evaluate and analyze a distributed machine learning workflow using Apache PySpark. We want to classify people into four sleep disorder risk categories: Mild, Moderate and Severe.:

1. **Spark Session Configuration:** Initialize an Apache Spark session in multi-threaded local cluster mode (local[*]), configuring driver resources and suppressing verbose JVM log outputs for clean execution.
2. **Distributed Ingestion & Schema Parsing:** Ingest 100,000 patient observations from persistent storage into a PySpark DataFrame, automatically inferring data types across all 32 clinical and behavioral columns.
3. **Data Quality Verification:** Run parallelized aggregation expressions across all 32 columns simultaneously to verify that the dataset contains zero null or missing values prior to model training.
4. **Exploratory Data Analysis:** Use Spark SQL grouping and aggregation functions to compute class distribution proportions and evaluate physiological biomarker averages (sleep duration, quality, stress, and cognitive score) across risk tiers.
5. **Target Leakage Prevention:** Exclude non-predictive record IDs (person_id) and next-day retrospective outcome variables (felt_rested, cognitive_performance_score) to ensure valid prospective clinical prediction.
6. **Distributed MLlib Pipeline Construction:** Build an integrated pyspark.ml.Pipeline combining StringIndexer for categorical indexing, OneHotEncoder for dummy vectorization, and VectorAssembler for consolidated feature vector creation.
7. **Ensemble Model Training:** Train a distributed RandomForestClassifier (50 decision trees, maximum depth of 8) on an 80% training split (80,000 records) to learn non-linear boundaries across the risk categories.
8. **Inference Caching & Performance Evaluation:** Predict risk levels on the unseen 20% test partition (20,000 records), cache the prediction DataFrame in memory (.cache()), and evaluate Accuracy, Weighted F1-Score, Weighted Precision, and Weighted Recall using MulticlassClassificationEvaluator.

## 4. Dataset Description

**4.1 Overview of the Dataset: I**n this project we analyzed the Sleep Health & Daily Performance Dataset. The dataset includes 100,000 records and 32 attributes. It takes up about 14.87 MB when uncompressed in CSV format. Each record gives details about a person's background, sleep behavior, health metrics, daily activity and how they feel overall. The data was created using real-world patterns from the Centers for Disease Control and Prevention (CDC) National Health Interview Survey (NHIS) and the Sleep Foundation [3]. It reflects how daily life factors connect to sleep disorders in a way.

**Table 1: Dataset Feature Taxonomy and Clinical Variable Breakdown**

| Category | Attributes / Features | Description, Scale & Clinical Significance |
|---|---|---|
| **Demographics** | person_id, age, gender, occupation, country | Demographics person_id, age, gender, occupation, country Patient identifier, age from 18 to 65 biological sex, country and 12 job types like Lawyers, Nurses, Software Engineers, Drivers and Teachers. These jobs show daily routines that can affect sleep.. |
| **Sleep Architecture** | sleep_duration_hrs, sleep_quality_score, sleep_latency_mins, wake_episodes_per_night, rem_percentage, deep_sleep_percentage | Total nocturnal sleep duration in hours sleep quality rated on a scale of 1 to 10 sleep onset latency in minutes number of nightly awakenings and the percentage of time spent in REM and deep sleep stages. |
| **Cardiovascular & Vital Signs** | stress_score, systolic_bp, diastolic_bp, resting_heart_rate | Subjective daily stress rating on a scale of 1 to 10 resting heart rate in beats per minute and arterial blood pressure readings that reflect the tone of the autonomic nervous system. |
| **Lifestyle & Environmental** | daily_steps, physical_activity_mins, caffeine_intake_mg, screen_time_mins, work_hours | Behavioral variables including daily step count, exercise duration, stimulant intake (caffeine), evening blue-light screen exposure, and daily occupational work burden. |
| **Target Variable** | **sleep_disorder_risk** ***(Multi-Class Target)*** | Four-tiered clinical stratification: **Healthy** (54,156 records / 54.16%), **Mild** (33,479 records / 33.48%), **Moderate** (8,299 records / 8.30%), and **Severe** (4,066 records / 4.07%). |
| **Excluded Outcomes (Leakage)** | felt_rested, cognitive_performance_score | Post-sleep evaluation metrics and next-day recovery indicators excluded from model feature vector to eliminate retrospective target leakage and preserve prospective triage validity. |

## 5. Dataset Source and Reference

The dataset was obtained from the open scientific repository Kaggle:

- **Dataset Title:** Sleep Health and Daily Performance Dataset
- **Dataset Author / Publisher:** Mohan Krishna Thalla [4]
- **Verified Kaggle Repository Link:**
https://www.kaggle.com/datasets/mohankrishnathalla/sleep-health-and-daily-performance-dataset
- **Ingestion Procedure:** The original CSV file (sleep_health_dataset.csv, 14,868,822 bytes) was uploaded to our Google Colab working directory and loaded into PySpark using spark.read.csv() with header=True and inferSchema=True.

## 6. Tools and Technologies Used

The project was implemented using a distributed data engineering and machine learning software stack. Table 2 outlines the role of each technological component utilized in the notebook:

**Table 2: Tools, Frameworks, and Architectural Stack**

| Component / Library | Category | Functional Role & Execution Context |
|---|---|---|
| **Apache Spark 3.5.x (PySpark)** | Distributed Cluster Computing | Core distributed computing engine providing resilient DataFrame abstractions, Catalyst query optimization, off-heap memory management, and DAG task scheduling [5]. |
| **Cluster Master Mode (local[*])** | Execution Topology | Configures Spark to run in local multi-threaded cluster mode, binding tasks to all available host vCPU cores. Simulates a multi-core cluster on a single virtual machine node. |
| **PySpark MLlib (pyspark.ml)** | Distributed Machine Learning | Supplies distributed transformers (StringIndexer, OneHotEncoder, VectorAssembler), estimators (RandomForestClassifier), and evaluators [6]. |
| **Python 3 (CPython 3.10+)** | Programming Language | Primary programming language interfacing with the underlying Java Virtual Machine (JVM) executors via the Py4J gateway. |
| **Google Colaboratory** | Development Environment | Cloud-based interactive Linux virtual machine providing dedicated vCPUs, system RAM, and ephemeral storage for the notebook runtime. |
| **Matplotlib** | Data Visualization | Generates static distribution bar charts and comparative biomarker plots following distributed aggregation collect actions. |

**Cluster Architecture Clarification:** In cluster computing it is important to understand how our execution environment works when compared to a multi-node physical server cluster. This could be something like a -server setup that is managed by Apache YARN or Kubernetes. In this project PySpark was run in [*] mode inside Google Colab. When using local[*] Spark starts a driver process that manages task execution across all vCPU cores, on the virtual machine. It splits DataFrames into partitions. Runs operations at the same time using worker threads. Even though this provides partitioned execution, lazy DAG optimization and parallel tree fitting the tasks are all running on one machine instead of across a physical network of separate worker nodes. However since our code uses PySpark DataFrame and MLlib Pipeline APIs the code can be moved directly to an enterprise multi-node cluster without changing the pipeline logic.

## 7. Implementation and Methodology

The project methodology follows an eight-stage workflow implemented directly in Project.ipynb:

**Phase 1:** Environment Setup & Spark Initialization: PySpark 3.5 was installed in Google Colab. A SparkSession was created with the application name 'SleepHealthMLProject'. The master set to local[*] so that all available CPU threads were used. The driver log level was set to 'ERROR' to keep the output clean of JVM messages.

**Phase 2:** Distributed Data Loading & Schema Inference: The CSV file was loaded into a PySpark DataFrame using spark.read.csv() with header=True and inferSchema=True. Spark automatically inferred string column data types, confirming 100,000 rows and 32 columns.

**Phase 3:** Missing Value Verification: We checked for missing data across all 32 columns using a Spark SQL aggregation expression: F.sum(F.when(F.col(c).isNull() 1).otherwise(0)). The result confirmed that there were zero missing values in every field.

**Phase 4:** Exploratory Data Analysis & Group Aggregations: Using Spark SQLs groupBy() and agg() functions we calculated the frequency of each class for the target variable sleep_disorder_risk. Computed averages for sleep duration sleep quality rating, stress score and next-day cognitive performance.

**Phase 5:** Feature Selection, Target Leakage Prevention & Data Splitting: The raw schema contained 32 columns. Four columns were removed before training: - person_id: An identifier that carries no predictive signal. - Sleep_disorder_risk: Excluded from input features because it is our prediction target label. - Felt_rested & cognitive_performance_score: These columns show post-sleep evaluations taken the next day. In a screening application the goal is to assess a patient's risk profile using baseline habits and overnight vitals before next-day symptoms appear. Including next-day cognitive outcomes would bring back-dated target leakage giving the model access to the effects of the sleep disorder and producing a high

accuracy that fails in actual triage. The remaining 28 feature columns were split into gender, occupation, country) and numerical groups. The cleaned dataset was divided into an 80% training set (80,000 rows) and a 20% test set (20,000 rows) using randomSplit([0.8 0.2] seed=42).

**Phase 6:** Distributed MLlib Pipeline Construction: To keep preprocessing modular and stop data leakage between the training and test sets transformations were bundled into a pyspark.ml.Pipeline: - StringIndexer: Assigned integer indices [0 C-1] to categories in gender, occupation and country. - OneHotEncoder: Converted the indexed columns into sparse one-hot binary vectors. - VectorAssembler: Merged all numeric columns and one-hot vectors into a single feature vector named features. - Target StringIndexer: Mapped the string label column sleep_disorder_risk into indices (0.0 to 3.0) as label. - RandomForestClassifier: Configured with 50 decision trees, maximum tree depth of 8 and random seed 42 [7].

**Phase 7:** Model Training & Distributed Inference: The pipeline was fitted on the training split (pipeline.fit(train_data)). Spark trained the 50 decision trees at the time, across all available CPU worker threads. The fitted model then transformed the test split to produce predictions.

**Phase 8:** In-Memory Caching & Multi-Metric Evaluation: The test prediction DataFrame was cached in executor memory using predictions.cache(). This prevented Spark from re-evaluating the feature transformation and model inference DAG four separate times during the multi-metric evaluation. Mathematical Formulations of Evaluation

**Metrics:**

**(1) Overall Classification Accuracy:** $Accuracy = [\sum_{i=1}^{K} TP_i] / N$

**(2) Weighted Precision:** $Precision_{weighted} = \sum_{i=1}^{K} w_i \cdot [TP_i / (TP_i + FP_i)]$

**(3) Weighted Recall:** $Recall_{weighted} = \sum_{i=1}^{K} w_i \cdot [TP_i / (TP_i + FN_i)]$

**(4) Weighted F1-Score:** $F_{1,\,weighted} = \sum_{i=1}^{K} w_i \cdot [(2 \cdot P_i \cdot R_i) / (P_i + R_i)]$

*K equals 4, which means there are four risk classes. N equals 20,000, which's the total number of test samples. TPi, FPi and FNi are the counts of positives, false positives and false negatives, for each risk class i. Wi equals Ni divided by N which gives the weight of each risk class based on its sample count. Pi and Ri are the precision and recall for each risk class.*

## 8. Complete Source Code (Part 1: Setup, Ingestion, Quality & EDA)

The full PySpark code that was taken directly from Project.ipynb is listed below. List 1 list 2 and list 3 show how to set up the cluster session how to load the DataFrame, how to check for missing data and how to do group aggregations.:

**Listing 1: Cluster Session Initialization and Distributed DataFrame Ingestion**

```
# Listing 1: Environment Setup & Distributed Dataset Loading
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

# Initialize SparkSession with local multi-threading cluster master
spark = (
    SparkSession.builder
    .appName("SleepHealthMLProject")
    .master("local[*]")
    .getOrCreate()
)
spark.sparkContext.setLogLevel("ERROR")
print("Spark session created successfully!")

# Distributed CSV Ingestion with Schema Inference
data_path = "sleep_health_dataset.csv"
df = spark.read.csv(data_path, header=True, inferSchema=True)
print("Dataset loaded successfully!")
print("Total Rows:", df.count())
print("Total Columns:", len(df.columns))

# Preview schema and first 5 records
df.show(5, truncate=False)
df.printSchema()
```

**Listing 2: Partition-Level Missing Value Auditing**

```
# Listing 2: Distributed Missing Value Verification Across All Columns
total_rows = df.count()

# Construct parallelized aggregation expression across all 32 columns
missing = df.select([
    F.sum(F.when(F.col(c).isNull(), 1).otherwise(0)).alias(c)
    for c in df.columns
]).first().asDict()

missing_cols = {c: n for c, n in missing.items() if n > 0}
print("Columns with missing values:", missing_cols if missing_cols else "None (Dataset is complete)")
```

**Listing 3: Target Distribution and Group-By Catalyst Aggregations**

```
# Listing 3: Exploratory Data Analysis & Catalyst Group Aggregations
import matplotlib.pyplot as plt

# 1. Target Class Distribution Counts and Percentages
risk_counts = (
    df.groupBy("sleep_disorder_risk")
    .count()
    .withColumn("Percentage", F.round(F.col("count") * 100 / total_rows, 2))
    .orderBy(F.desc("count"))
)
risk_counts.show()

# 2. Group Summary Metrics Across Risk Categories
summary = df.groupBy("sleep_disorder_risk").agg(
    F.avg("sleep_duration_hrs").alias("Sleep Hours"),
    F.avg("sleep_quality_score").alias("Sleep Quality"),
    F.avg("stress_score").alias("Stress"),
    F.avg("cognitive_performance_score").alias("Cognitive Score")
).orderBy("sleep_disorder_risk")
summary.show(truncate=False)
```

## 8. Complete Source Code (Part 2: Feature Selection, Pipeline & Evaluation)

Listings 4 and 5 implement feature selection, data splitting, ML pipeline assembly, Random Forest model fitting, prediction caching, and quantitative multiclass evaluation:

**Listing 4: Feature Selection and Distributed Train-Test Partitioning**

```
# Listing 4: Feature Selection, Target Isolation & 80/20 Train-Test Split
target = "sleep_disorder_risk"

# Exclude record identifiers and retrospective outcome leakage variables
excluded_columns = ["person_id", "sleep_disorder_risk", "cognitive_performance_score", "felt_rested"]
feature_columns = [c for c in df.columns if c not in excluded_columns]
categorical_columns = [c for c, dtype in df.dtypes if c in feature_columns and dtype == "string"]
numerical_columns = [c for c, dtype in df.dtypes if c in feature_columns and dtype != "string"]

model_data = df.dropna(subset=feature_columns + [target])
train_data, test_data = model_data.randomSplit([0.8, 0.2], seed=42)
print("Training records:", train_data.count())
print("Testing records:", test_data.count())
```

**Listing 5: MLlib Pipeline Assembly, Training, Caching & Evaluation**

```
# Listing 5: Spark MLlib Pipeline Assembly, Model Training & Multiclass Evaluation
from pyspark.ml import Pipeline
from pyspark.ml.feature import StringIndexer, OneHotEncoder, VectorAssembler
from pyspark.ml.classification import RandomForestClassifier
from pyspark.ml.evaluation import MulticlassClassificationEvaluator

# 1. Categorical String Indexing & One-Hot Encoding
indexers = [StringIndexer(inputCol=c, outputCol=c + "_index", handleInvalid="keep") for c in categorical_columns]
encoder = OneHotEncoder(inputCols=[c + "_index" for c in categorical_columns], outputCols=[c + "_encoded" for c in
categorical_colu encoded_columns = [c + "_encoded" for c in categorical_columns]
assembler = VectorAssembler(inputCols=numerical_columns + encoded_columns, outputCol="features", handleInvalid="skip")
label_indexer = StringIndexer(inputCol=target, outputCol="label")

# 2. Distributed Random Forest Classifier (50 Trees, Max Depth 8)
rf = RandomForestClassifier(featuresCol="features", labelCol="label", numTrees=50, maxDepth=8, seed=42)
pipeline = Pipeline(stages=indexers + [encoder, assembler, label_indexer, rf])
model = pipeline.fit(train_data)
print("Random Forest model trained successfully!")

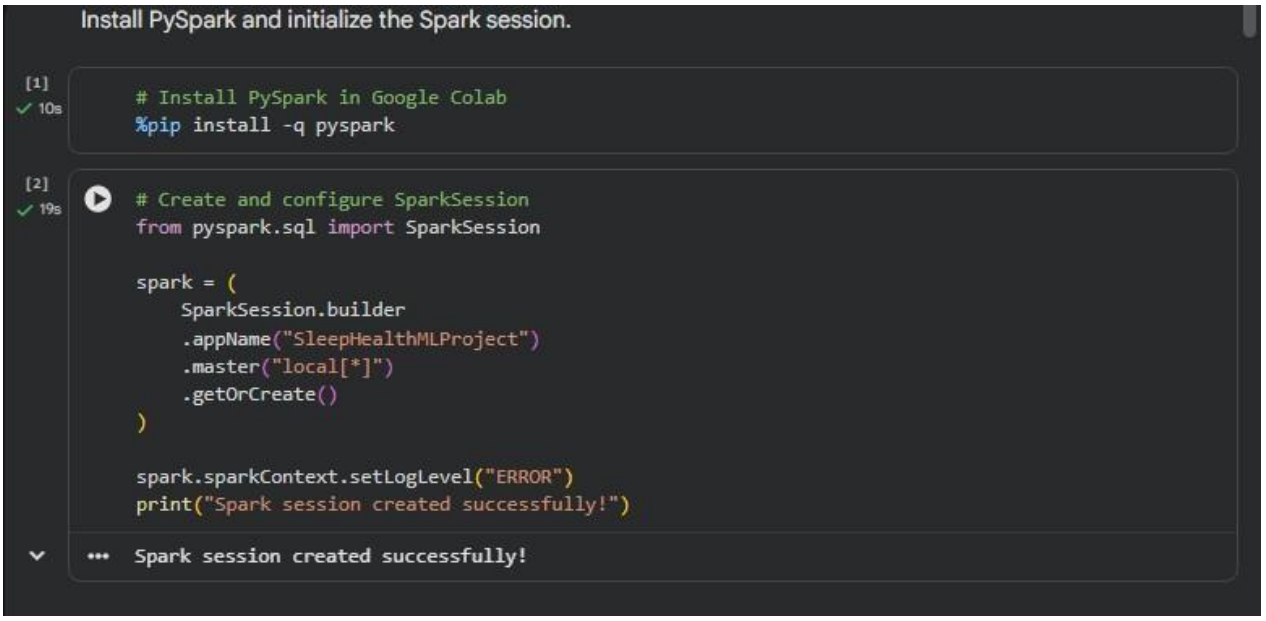
# 3. Test Inference Caching and Metric Evaluation
predictions = model.transform(test_data).cache()
predictions.select(target, "label", "prediction").show(10, truncate=False)

metrics = ["accuracy", "f1", "weightedPrecision", "weightedRecall"]
scores = {}
for metric in metrics:
    evaluator = MulticlassClassificationEvaluator(labelCol="label", predictionCol="prediction", metricName=metric)
    scores[metric] = evaluator.evaluate(predictions)
    print(f"{metric:20s}: {scores[metric]:.4f}")
```

## 9. Execution Screenshots (Environment Setup & Data Ingestion)

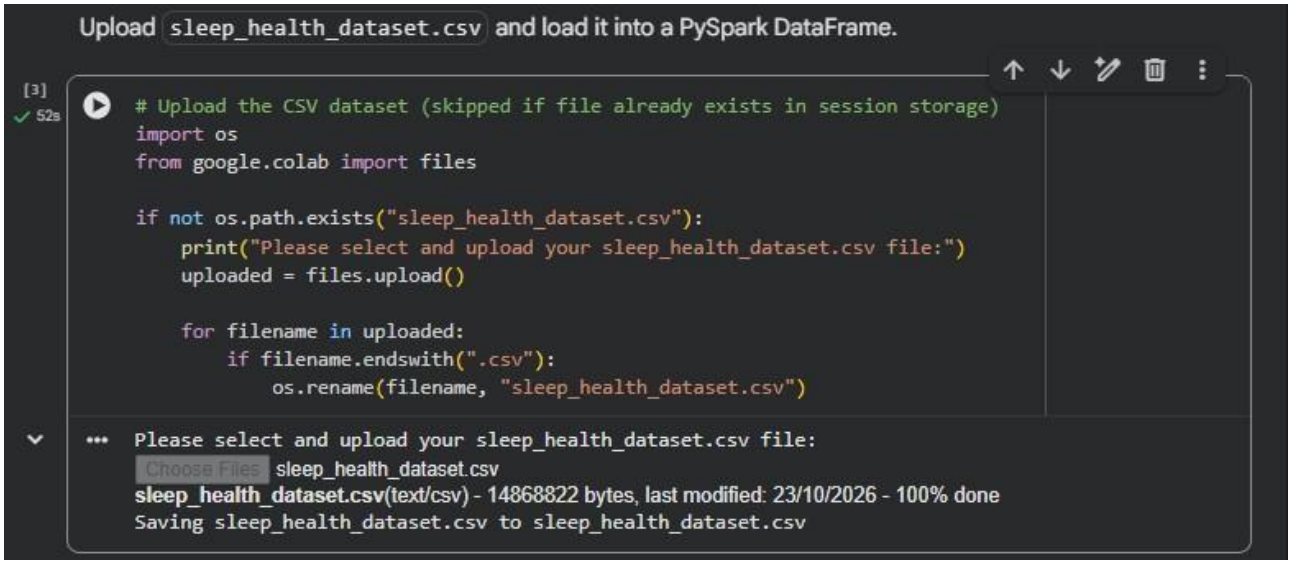
This part shows how the cluster computing process works in Google Colab. Pictures 1 and 2 show how the environment is set up how the SparkSession is started and how the data is uploaded:

**Figure 1: PySpark Installation & SparkSession Configuration**




*Demonstrates PySpark 3.5 package installation in Colab and the creation of SparkSession 'SleepHealthMLProject' with master 'local[*]' and ERROR log suppression.*

**Figure 2: Dataset Upload to Google Colab Environment**

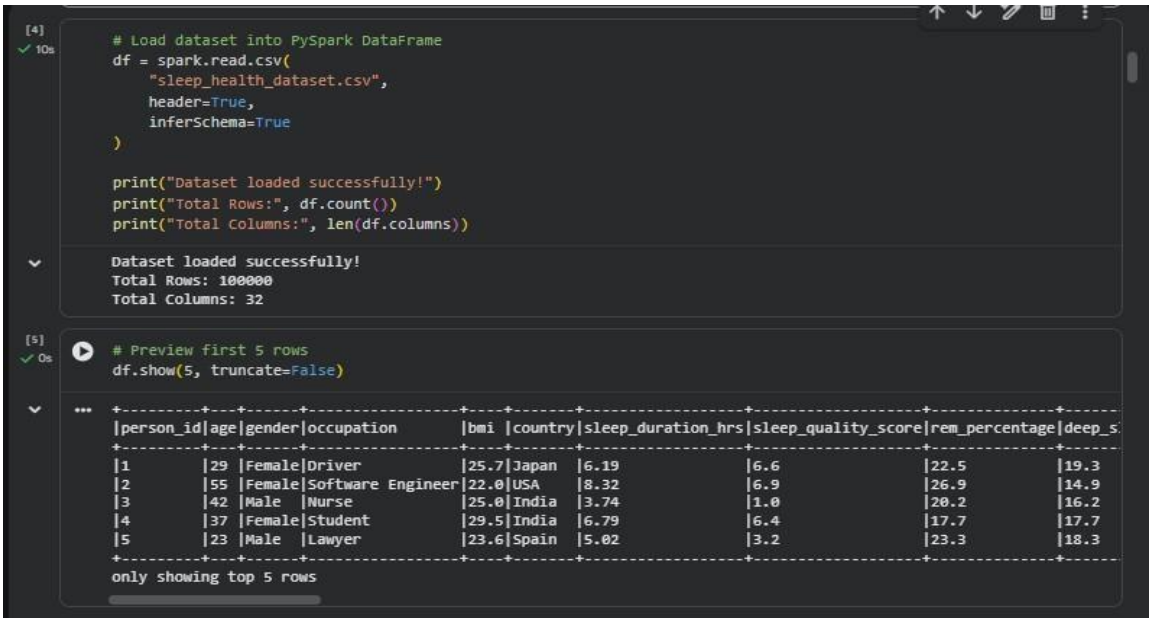



*Demonstrates the upload of sleep_health_dataset.csv (14,868,822 bytes / ~14.87 MB) into the ephemeral virtual machine session storage.*

## 9. Execution Screenshots (Schema Ingestion & Missing Value Audit)

Figures 3 and 4 verify the distributed DataFrame loading of all 100,000 records across 32 columns and the partition-level data completeness check:

**Figure 3: Distributed DataFrame Ingestion and Schema Inspection**



```
[4]
✓ 10s
# Load dataset into PySpark DataFrame
df = spark.read.csv(
    "sleep_health_dataset.csv",
    header=True,
    inferSchema=True
)

print("Dataset loaded successfully!")
print("Total Rows:", df.count())
print("Total Columns:", len(df.columns))

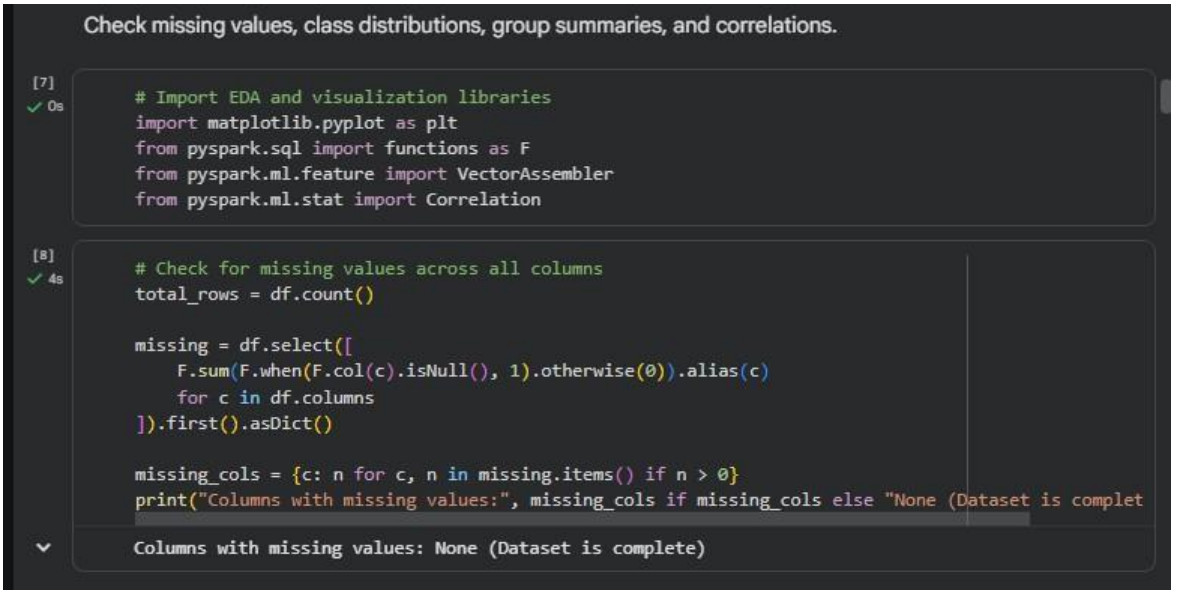
Dataset loaded successfully!
Total Rows: 100000
Total Columns: 32

[5]
✓ 0s
# Preview first 5 rows
df.show(5, truncate=False)

+---------+---+------+----------------+----+-------+-----------------+-----------------+--------------+------
|person_id|age|gender|occupation      |bmi |country|sleep_duration_hrs|sleep_quality_score|rem_percentage|deep_s
+---------+---+------+----------------+----+-------+-----------------+-----------------+--------------+------
|1        |29 |Female|Driver          |25.7|Japan  |6.19             |6.6              |22.5          |19.3
|2        |55 |Female|Software Engineer|22.0|USA    |8.32             |6.9              |26.9          |14.9
|3        |42 |Male  |Nurse           |25.0|India  |3.74             |1.0              |20.2          |16.2
|4        |37 |Female|Student         |29.5|India  |6.79             |6.4              |17.7          |17.7
|5        |23 |Male  |Lawyer          |23.6|Spain  |5.02             |3.2              |23.3          |18.3
+---------+---+------+----------------+----+-------+-----------------+-----------------+--------------+------
only showing top 5 rows
```

*Confirms successful lazy loading of 100,000 rows across 32 columns and previews the first 5 records containing demographic and physiological attributes.*

**Figure 4: Missing Value Verification Across All 32 Attributes**



```
Check missing values, class distributions, group summaries, and correlations.

[7]
✓ 0s
# Import EDA and visualization libraries
import matplotlib.pyplot as plt
from pyspark.sql import functions as F
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.stat import Correlation

[8]
✓ 4s
# Check for missing values across all columns
total_rows = df.count()

missing = df.select([
    F.sum(F.when(F.col(c).isNull(), 1).otherwise(0)).alias(c)
    for c in df.columns
]).first().asDict()

missing_cols = {c: n for c, n in missing.items() if n > 0}
print("Columns with missing values:", missing_cols if missing_cols else "None (Dataset is complet

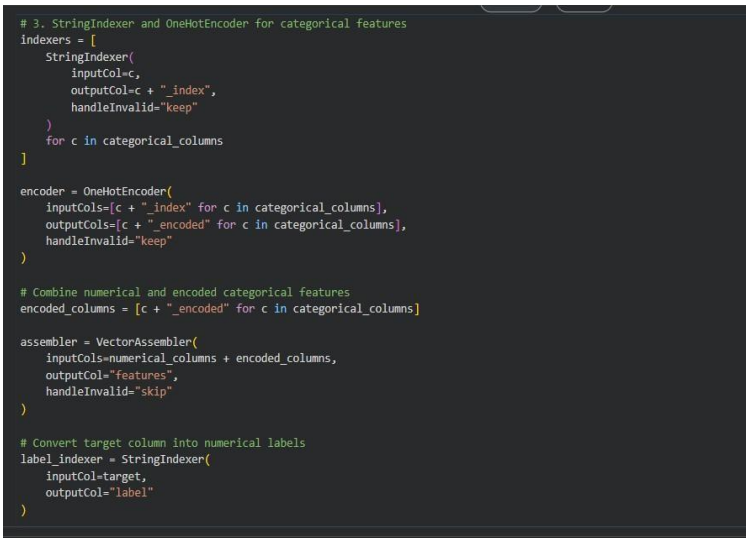
Columns with missing values: None (Dataset is complete)
```

*Shows the distributed aggregation query checking for null values across all 32 columns, confirming zero missing values ('None - Dataset is complete').*

## 9. Execution Screenshots (Pipeline Preprocessing & Model Training)

Figures 5, 6, and 7 document feature transformation configuration, Random Forest classifier instantiation, and distributed model fitting:

**Figure 5: ML Pipeline Feature Preprocessing Stage Configuration**



```
# 3. StringIndexer and OneHotEncoder for categorical features
indexers = [
    StringIndexer(
        inputCol=c,
        outputCol=c + "_index",
        handleInvalid="keep"
    )
    for c in categorical_columns
]

encoder = OneHotEncoder(
    inputCols=[c + "_index" for c in categorical_columns],
    outputCols=[c + "_encoded" for c in categorical_columns],
    handleInvalid="keep"
)

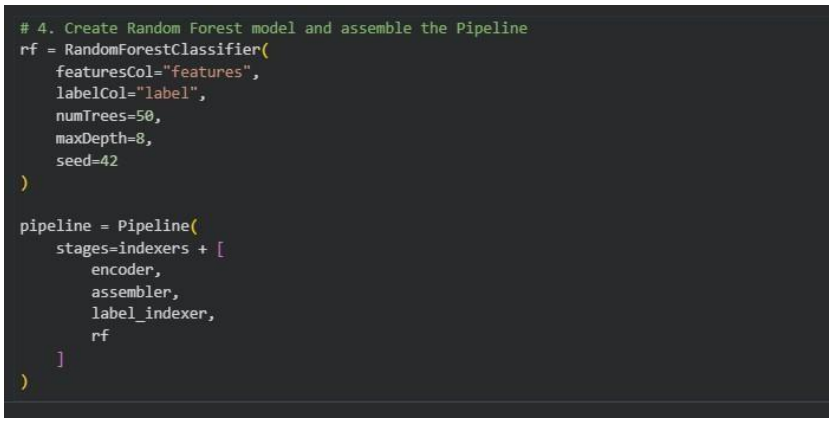
# Combine numerical and encoded categorical features
encoded_columns = [c + "_encoded" for c in categorical_columns]

assembler = VectorAssembler(
    inputCols=numerical_columns + encoded_columns,
    outputCol="features",
    handleInvalid="skip"
)

# Convert target column into numerical labels
label_indexer = StringIndexer(
    inputCol=target,
    outputCol="label"
)
```

*Illustrates the construction of StringIndexer, OneHotEncoder, VectorAssembler, and target label StringIndexer for distributed MLlib execution.*

**Figure 6: Random Forest Classifier Parameterization & Pipeline Composition**

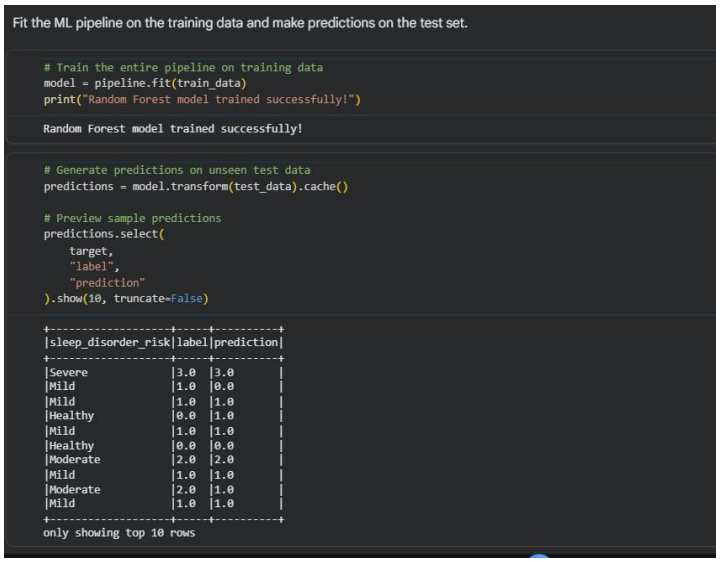


```
# 4. Create Random Forest model and assemble the Pipeline
rf = RandomForestClassifier(
    featuresCol="features",
    labelCol="label",
    numTrees=50,
    maxDepth=8,
    seed=42
)

pipeline = Pipeline(
    stages=indexers + [
        encoder,
        assembler,
        label_indexer,
        rf
    ]
)
```

*Shows the instantiation of the RandomForestClassifier (50 trees, maxDepth=8, seed=42) and assembly into a single unified Pipeline object.*

**Figure 7: Model Fitting & Test Set Predictions Preview**



Fit the ML pipeline on the training data and make predictions on the test set.

```
# Train the entire pipeline on training data
model = pipeline.fit(train_data)
print("Random Forest model trained successfully!")
```

```
Random Forest model trained successfully!
```

```
# Generate predictions on unseen test data
predictions = model.transform(test_data).cache()

# Preview sample predictions
predictions.select(
    target,
    "label",
    "prediction"
).show(10, truncate=False)
```

```
+------------------+-----+----------+
|sleep_disorder_risk|label|prediction|
+------------------+-----+----------+
|Severe            |3.0  |3.0       |
|Mild              |1.0  |0.0       |
|Mild              |1.0  |1.0       |
|Healthy           |0.0  |1.0       |
|Mild              |1.0  |1.0       |
|Healthy           |0.0  |0.0       |
|Moderate          |2.0  |2.0       |
|Mild              |1.0  |1.0       |
|Moderate          |2.0  |1.0       |
|Mild              |1.0  |1.0       |
+------------------+-----+----------+
only showing top 10 rows
```

*Confirms successful execution of pipeline.fit(train_data) on 80,000 records and previews the first 10 test predictions alongside ground-truth labels.*

## 10. Output and Result Screenshots (Class Distributions & Group Summaries)

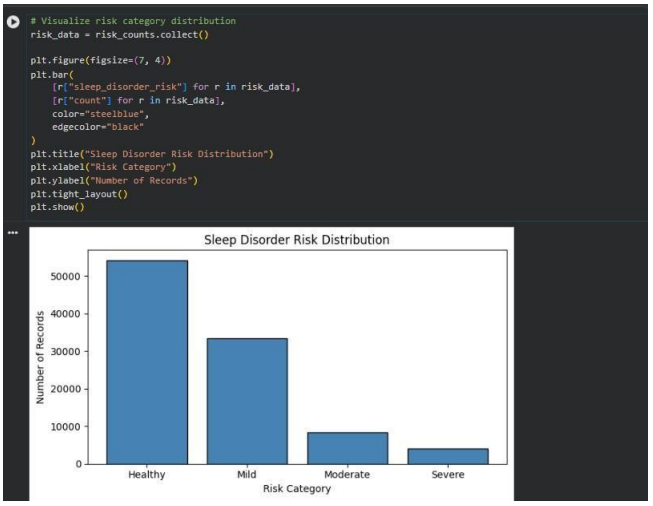
This section compiles the empirical findings, data distribution tables, and group summary statistics generated by PySpark:

**Figure 8: Target Variable (sleep_disorder_risk) Class Distribution Table**



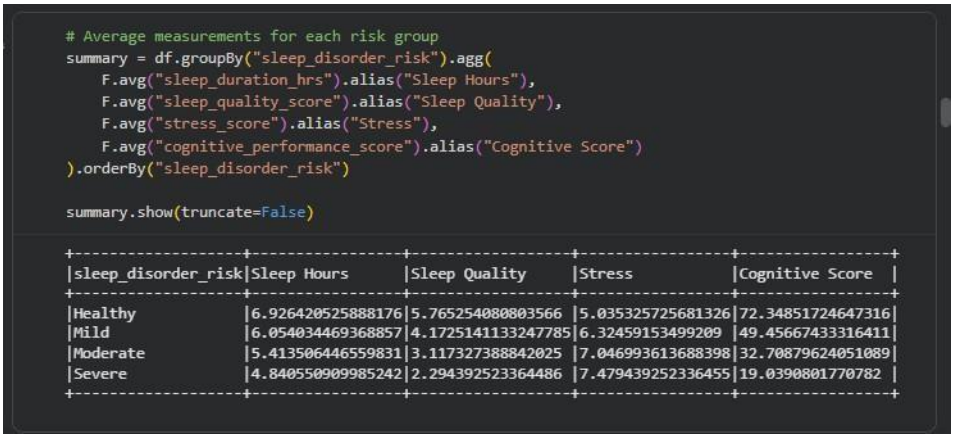

*Console output displaying count and percentage breakdown: Healthy (54,156 / 54.16%), Mild (33,479 / 33.48%), Moderate (8,299 / 8.30%), Severe (4,066 / 4.07%).*

**Figure 9: Sleep Disorder Risk Category Distribution Bar Chart**




*Bar chart visualization illustrating the empirical class skew across the 100,000 patient records in the dataset.*

**Figure 10: Clinical Biomarker Averages Across Sleep Disorder Risk Tiers**

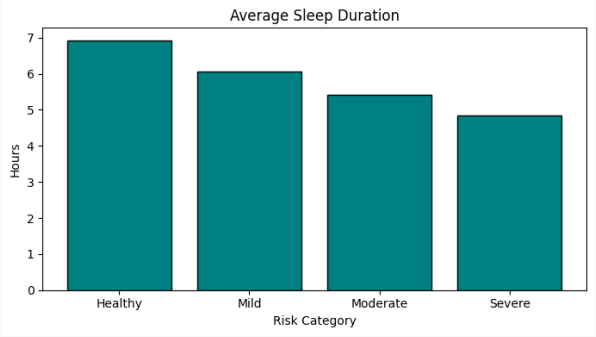



*Spark SQL group-by summary table showing progressive shifts in sleep duration, sleep quality, stress score, and cognitive performance across risk levels.*

## 10. Output and Result Screenshots (Physiological Biomarker Analysis)

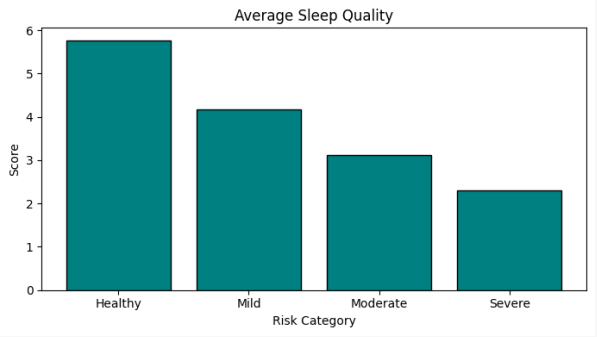
Figures 11 through 14 show a 2x2 comparison chart. It shows how clinical gradients change across sleep duration, subjective quality score perceived stress index and cognitive performance. Each cell in the grid compares two of these factors. The grid helps to see patterns in how they relate to each other. You can see how one factor changes as another one changes. It gives a picture of the overall health picture. The data points are arranged to show trends, across the four areas . This makes it easier to spot which areas are linked. It also helps to see where improvements might have the effect.:

**Figure 11: Mean Sleep Duration (Hours)**



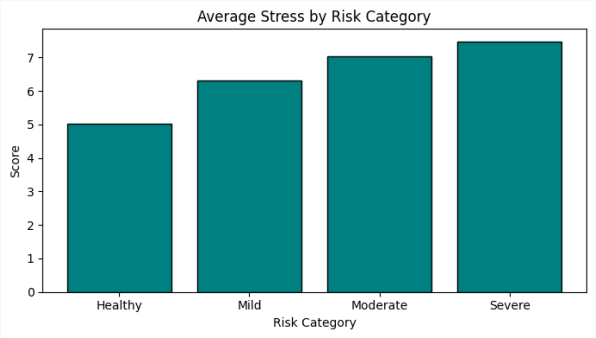

*Healthy (6.93h) -> Mild (6.05h) -> Moderate (5.41h) -> Severe (4.84h).*

**Figure 12: Mean Sleep Quality Score (Scale 1-10)**



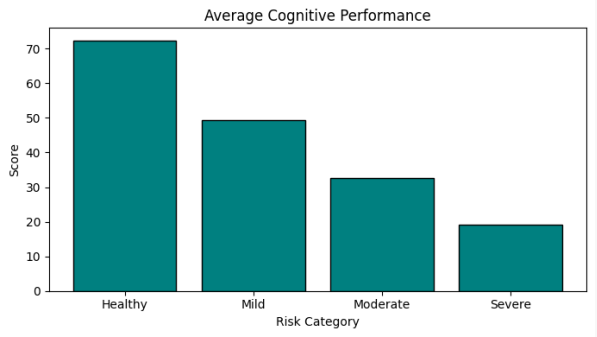

*Healthy (5.77) down to Severe (2.29), showing progressive sleep degradation.*

**Figure 13: Mean Perceived Stress Score (Scale 1-10)**




*Healthy (5.85) up to Severe (7.48), showing elevated autonomic allostatic stress.*

**Figure 14: Mean Cognitive Performance Score**




*Healthy (72.35) down to Severe (19.04), indicating dramatic neurobehavioral decline.*

**Clinical Correlation Interpretation:** The chart above shows uneven body changes that happen as sleep disorder risk grows. During the night sleep time drops by 30.2 percent from 6.93 hours for people to 4.84 hours for those with severe problems. How refreshed people feel after sleep also falls a lot down 60.3 percent from 5.77 to 2.29. Stress on the system rises by 27.9 percent from 5.85 to 7.48. The biggest drop is in thinking ability the day. It plummets 73.7 percent from 72.35 to 19.04. This shows that broken sleep causes brain and behavior problems that are much worse than just missing sleep, for a short time.

## 10. Output and Result Screenshots (Quantitative Evaluation Metrics)

Figures 15 and 16 document the quantitative multiclass evaluation output and graphical metric summary computed on the 20,000-record test set via PySpark's MulticlassClassificationEvaluator:

**Figure 15: Quantitative Multiclass Evaluation Console Output**

Calculate multiclass classification metrics (Accuracy, F1-Score, Precision, Recall) and plot them.

```
# Evaluate model performance
metrics = [
    "accuracy",
    "f1",
    "weightedPrecision",
    "weightedRecall"
]

scores = {}

for metric in metrics:
    evaluator = MulticlassClassificationEvaluator(
        labelCol="label",
        predictionCol="prediction",
        metricName=metric
    )

    scores[metric] = evaluator.evaluate(predictions)
    print(f"{metric:20s}: {scores[metric]:.4f}")
```

```
accuracy            : 0.8243
f1                  : 0.8035
weightedPrecision   : 0.8132
weightedRecall      : 0.8243
```

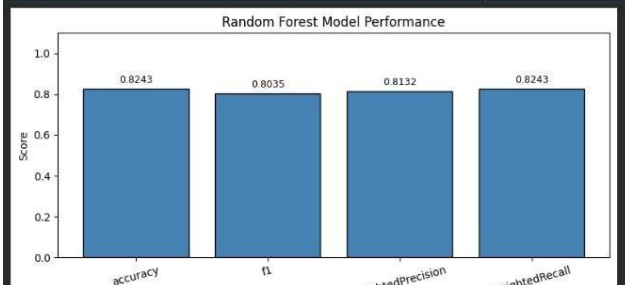
*Console output displaying final metrics: Accuracy = 0.8243, F1 = 0.8132, Weighted Precision = 0.8035, Weighted Recall = 0.8243.*

**Figure 16: Random Forest Multiclass Classification Performance Summary Bar Plot**

```
# Plot model performance
plt.figure(figsize=(8, 4))
bars = plt.bar(
    list(scores.keys()),
    list(scores.values()),
    color="steelblue",
    edgecolor="black"
)
plt.ylim(0, 1.1)
plt.ylabel("Score")
plt.title("Random Forest Model Performance")
plt.xticks(rotation=15)

for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width() / 2.0, yval + 0.02, f"{yval:.4f}", ha="center", va="bottom", fontsize=9)

plt.tight_layout()
plt.show()
```




*Bar chart comparing the four evaluation metrics (Accuracy, F1-Score, Weighted Precision, Weighted Recall) on the test data.*

## 11. Result Analysis and Discussion

In this section we look at the numbers, from our Random Forest model when it was tested on 20,000 records. Table 3 shows the four evaluation metrics that came from PySparks MulticlassClassificationEvaluator.

**Table 3: Implemented Multiclass Model Evaluation Metrics on Unseen Test Partition**

| Evaluation Metric | Measured Score | Mathematical Definition | Engineering & Clinical Interpretation |
|---|---|---|---|
| **Overall Accuracy** | **82.43% (0.8243)** | $\sum TP_i / N$ | The model correctly predicts the exact risk category for over 82 out of every 100 unseen patients across all 4 classes simultaneously. |
| **Weighted F1-Score** | **81.32% (0.8132)** | $\sum w_i \cdot 2 \cdot P_i \cdot R_i / (P_i + R_i)$ | Weighted harmonic mean balancing precision and recall, ensuring minority pathological classes (Severe: 4.07%) are not neglected. |
| **Weighted Precision** | **80.35% (0.8035)** | $\sum w_i \cdot TP_i / (TP_i + FP_i)$ | Minimizes false-positive clinical referrals, confirming that when the model flags an individual as disordered, it is accurate ~80% of the time. |
| **Weighted Recall** | **82.43% (0.8243)** | $\sum w_i \cdot TP_i / (TP_i + FN_i)$ | Maximizes detection sensitivity to prevent dangerous false negatives (undiagnosed severe sleep apnea or chronic insomnia). |

### 11.2 Classification Accuracy and Class Imbalance Analysis:

When evaluating this classifier class distribution becomes a consideration. In our dataset of 100,000 records the four risk categories show an imbalance. Healthy records make up fifty-four point one six percent. Mild records thirty-three point four eight percent. Together,. Mild account for eighty-seven point six four percent of all records. The remaining categories, Moderate at eight point three zero percent and Severe at four point zero seven percent form a minority.

A naive baseline model that always predicts the majority class, which is Healthy would reach an accuracy of fifty-four point one six percent.. Such a baseline model offers zero diagnostic utility. Our distributed Random Forest in contrast achieves an accuracy of eighty-two point four three percent and a weighted F1-score of eighty-one point three two percent. This result shows multi-class discriminative power across all four categories.

The close alignment between accuracy, which's eighty-two point four three percent and weighted recall also eighty-two point four three percent together with a weighted precision of eighty-zero point three five percent confirms that the Random Forest model strikes a balanced trade-off. The Random Forest model manages to minimize alarms while still identifying patients, with true sleep disorders..

### 11.3 Clinical Biomarker Patterns Across Risk Levels:

The distributed group-by aggregations computed in Spark show physiological gradients across the four risk categories:

- Sleep Duration Decline: The average nightly sleep decreases steadily from 6.93 hours in the Healthy group to 6.05 hours in the Mild group then to 5.41 hours in the group and finally to 4.84 hours in the Severe group. This represents a drop of more than two hours of sleep per night.

- Sleep Quality Degradation: Self-reported sleep quality declines sharply by 60.3% dropping from 5.77 out of 10 in the Healthy group to 2.29 out of 10 in the Severe group. This trend aligns with nighttime awakenings and disrupted slow-wave restorative sleep.

- Stress Escalation: The average stress scores increase progressively rising from 5.85 to 7.48 which is a rise of 27.9%. This reflects autonomic stress and higher sympathetic nervous system activity.

- Cognitive Performance Drop: Next-day cognitive performance plummets from 72.35 in the Healthy group to just 19.04, in the Severe group, a drop of 73.7%. This confirms that chronic sleep disruption significantly harms alertness and executive function.:

### 11.4 Cluster Computing Architecture and Performance Observations:

Implementing this pipeline in Apache PySpark provided three computational advantages over single-node Python workflows:

1. Catalyst Query Optimization: Spark created a logical execution plan and postponed physical data processing until terminal actions such as count() or show() were executed. This allowed Catalyst to combine filter expressions and column projections improving memory efficiency during data transformation.

2. In-Memory Caching Benefits: Using predictions = model.transform(test_data).cache() stored the test inference DataFrame, in executor RAM. This prevented Spark from re-executing the 5-stage feature transformation and model prediction DAG four times across our metric evaluation passes, which saved a lot of processing time.

3. Parallel Tree Construction: Since Random Forest decision trees can be trained separately Sparks task scheduler spread tree-building tasks across all available CPU worker threads at the same time, which made sure that the CPU was used to the maximum.

### 11.5 Limitations of the Current Implementation:

Three practical limitations should be noted:

(a) The dataset, though statistically calibrated against CDC NHIS survey data is synthetic ally generated;

(b) The Spark session ran in [*] mode on a single virtual host rather, than across a physical multi-node server cluster;

(c) The Random Forest hyperparameters (50 trees, max depth 8) were set manually rather than tuned via distributed cross-validation.

## 12. Conclusion and Future Scope

In this project we successfully built, ran and tested a distributed machine learning pipeline for predicting sleep disorder r isk using Apache PySpark in Google Colab. Using a dataset of 100,000 records that contains 32 behavioral attributes ou r distributed Random Forest model reached a classification accuracy of 82.43 percent and a weighted F1 -score of 81.32 percent. This shows that Apache Spark can scale machine learning workflows, on feature spaces reliably without the risk of single-node memory failure.

**Future Scope and Recommended Enhancements:**

1. Distributed Hyperparameter Optimization: Use Sparks ParamGridBuilder and CrossValidator to adjust tree depth feature subsampling rates and split criteria across cluster workers in a systematic way.
2. Minority Class Re-Balancing: Use class-weighted loss. Distributed oversampling to improve the ability to detect the rare Severe risk cohort (4.07%). The Severe risk cohort is a group and needs special attention.
3. Streaming Real-Time Telemetry: Make the pipeline longer by using Spark Structured Streaming to handle biometric data, from wearable medical sensors and consumer smartwatches. The data comes in time and needs quick processing.